



SESSION 09 · WEEK 3

Teaching KelanaAI to Read Knowledge

Build KelanaAI with Python, Next.js & Amazon Bedrock — Session 9



🕒 Duration: 2 Hours

📌 Week 3 · RAG with Knowledge Base

📌 Feature: Retrieval-Augmented Generation

Yesterday KelanaAI learned who its users are.

Today KelanaAI learns how to use trusted knowledge.



SFIA Competency Card

Session 9 — Teaching KelanaAI to Read Knowledge

FEATURE DELIVERED



Retrieval-Augmented Generation (RAG) with Amazon Bedrock Knowledge Bases

PRIMARY SFIA SKILLS

INAI L3

Artificial Intelligence Implementation

KNOW L2

Knowledge Management

PROG L2

Programming / Software Development

SWDN L2

Software Design

EVIDENCE — BY END OF SESSION, STUDENTS PRODUCE:

- ✓ Amazon Bedrock Knowledge Base
- ✓ Uploaded travel knowledge documents
- ✓ RAG-enabled API endpoint
- ✓ AI answers grounded in custom knowledge
- ✓ Git commit

```
rag · knowledge-base ·  
grounded-ai
```

Learning Objectives

By the end of this session, students will be able to:

01



Explain the limitations of LLMs.
Models don't know your private data.

02



Describe what RAG is.
Retrieve documents, then generate.

03



Create an Amazon Bedrock Knowledge Base.
A managed index over your docs.

04



Upload documents to a knowledge source.
PDFs, text files, travel guides.

05



Query a Knowledge Base from FastAPI.
RetrieveAndGenerate API call.

06



Compare standard LLM vs RAG responses.
General answer vs grounded answer.

Product Roadmap

Where we are — Week 3, RAG (completing the AI capabilities)

WEEK 1 ✓	WEEK 2 ✓	WEEK 3 · TODAY 🎯	WEEK 4 ⌚
✓ Trip Summary	✓ PostgreSQL	✓ Trip Dashboard	✓ Conversation History
✓ Recommendation Engine	✓ Amazon Bedrock	✓ Authentication	✓ Deployment
✓ REST API	✓ Next.js Frontend	▶ RAG with Knowledge Base	✓ Capstone



Yesterday KelanaAI learned who its users are. **Today** KelanaAI learns how to use trusted knowledge instead of relying only on what the AI model already knows.

Story — When the LLM Doesn't Know

Business applications need answers based on company policies, internal docs, and travel guides — not just general knowledge.

IMAGINE A TRAVELER ASKS

? "Do Indonesian passport holders need a visa to visit Japan?"

? "What baggage allowance does Sinaptik Travel provide?"

? Can a general LLM answer correctly?

Not necessarily.

The model wasn't trained on your company's policies or newly updated regulations.

Business apps need answers based on:

- ✓ Company policies
- ✓ Internal documents
- ✓ Product manuals
- ✓ Travel guides
- ✓ Visa requirements

This approach is called **Retrieval-Augmented Generation (RAG)**.

WITHOUT RAG — GENERAL LLM

Trained on public internet data.
May hallucinate. No access to
your docs.

RAG

WITH RAG — GROUNDED

Retrieves from your trusted docs
first. Answers are grounded and
citable.



Technical Lesson

Eight parts — from LLM limitations to a grounded RAG assistant.

01

LLM Limits

02

What is RAG

03

Create KB

04

Upload Docs

05

Query KB

06

Integrate

07

Compare

08

Next.js

Part 1 — Why LLM Knowledge Isn't Enough

The model cannot guarantee answers based on your company's policies or newly updated regulations.

◆ CURRENT ARCHITECTURE



User



Amazon Bedrock (LLM only)



Response



No access to your documents. **May hallucinate.**



THE PROBLEM



The model can't guarantee answers based on:

- ✗ Your company's policies
- ✗ Newly updated travel regulations
- ✗ Internal documentation

WHY?

LLMs are trained on public data up to a cutoff date. They don't know your private documents, and they can't verify facts against sources they've never seen.

THE FIX

Retrieval-Augmented Generation (RAG) — give the model your documents at query time.

Coming up in Part 2.

Part 2 — What is RAG?

Retrieval-Augmented Generation. The AI retrieves relevant documents first, then uses them to answer.

NEW ARCHITECTURE · RAG



User Question



Knowledge Base (retrieval)



Relevant Documents



Amazon Bedrock (LLM)



Grounded Answer

Steps 2-3 are new. The model reasons over retrieved context.

CONCEPT



Retrieve, then generate

The AI doesn't memorize your documents. Instead, it retrieves the most relevant passages first, then uses them as context to generate a grounded answer.

WHY IT WORKS

- ✓ Answers cite real sources
- ✓ Easy to update — just swap docs
- ✓ No need to retrain the model

vs FINE-TUNING

Fine-tuning teaches the model new patterns. RAG gives the model new facts at query time. RAG is cheaper, faster to update, and more transparent.

RAG = search + generation.

Part 3 — Create a Knowledge Base

Inside AWS: create a Bedrock Knowledge Base, connect it to S3 and Bedrock.

AWS SETUP

01

⊕ Create Knowledge Base

In AWS Console → Amazon Bedrock → Knowledge Bases → Create.

Name: `kelana-travel-kb`

02

↑ Connect Amazon S3

Choose an S3 bucket as the data source. Bedrock will index its contents.

s3://kelana-travel-docs/

03

⌘ Connect Amazon Bedrock

Select a foundation model (e.g. Claude) for generation. Choose an embedding model for retrieval.

Model: `anthropic.claude-3-sonnet`

AWS manages the vector store and indexing — no infrastructure to build.

KEY POINT



Your documents stay yours

The documents remain in your S3 bucket. The model simply references them during inference — it never trains on them permanently.

WHAT AWS MANAGES

- ✓ Vector embeddings (for search)
- ✓ Document chunking
- ✓ Retrieval at query time

DATA PRIVACY

Documents are not used to train the foundation model. They're only retrieved when your app queries the Knowledge Base.

Update docs → sync KB → answers update instantly.

Part 4 — Upload Documents

Create a folder of travel knowledge PDFs. Upload to S3. Sync the Knowledge Base.

```
travel-guides/

travel-guides/
├─ visa-japan.pdf           # visa requirements for Japan
├─ tokyo-attractions.pdf    # top sights in Tokyo
├─ packing-checklist.pdf    # what to pack for Japan
└─ travel-insurance.pdf     # coverage details
```

```
bash — upload to S3

# upload the folder to S3
$ aws s3 sync travel-guides/ \
s3://kelana-travel-docs/
# sync the Knowledge Base
$ aws bedrock start-ingestion-job ...
```

Whenever documents change, re-sync the Knowledge Base.

CONCEPT

Docs → S3 → Knowledge Base

Upload PDFs, text files, or Markdown to S3. Bedrock chunks them, generates embeddings, and indexes everything for fast retrieval.

SUPPORTED FORMATS

PDF

TXT

Markdown

DOCX

KEEP DOCS FRESH

- ✓ Update the PDF in S3
- ✓ Trigger a sync job
- ✓ New answers reflect the update instantly

No model retraining — just re-sync.

Part 5 — Query the Knowledge Base

Instead of `InvokeModel`, call `RetrieveAndGenerate`. Bedrock handles retrieval automatically.

```
services/kb_service.py

import boto3, json

client = boto3.client( "bedrock-agent-runtime" )

def ask_knowledge_base(question: str):
    response = client.retrieve(
        knowledgeBaseId=KNOWLEDGE_BASE_ID,
        retrievalQuery={"text": query},
        retrievalConfiguration={
            "managedSearchConfiguration": {
                "numberOfResults": 1,
            },
        },
    )

    return response["output"]["text"]
```

One API call — retrieval + generation handled by Bedrock.

RETRIEVAL FLOW



Bedrock does the search

Developers don't manually search documents. Amazon Bedrock performs retrieval automatically — it finds the relevant passages and passes them to the model.

WHAT HAPPENS INSIDE

- 1 Question received
- ↓
- 2 Knowledge Base searches
- ↓
- 3 Relevant chunks retrieved
- ↓
- 4 Foundation model generates
- ↓
- 5 Grounded answer returned

API COMPARISON

- `InvokeModel` — LLM only
- `Retrieve` — RAG

Same client, different method.

Part 6 — Integrate with KelanaAI

Create POST `/api/v1/ask`. The backend sends the question to the Knowledge Base and returns a grounded answer.

● ● ● POST `/api/v1/ask` — Request

REQUEST

```
{
  "question": "Do I need a visa to visit Japan?"
}
```

● ● ● main.py

```
from services.kb_service import ask_knowledge_base

@app.post("/api/v1/ask")
def ask_endpoint(request: QuestionRequest):
    # 1. Send question to Knowledge Base
    answer = ask_knowledge_base(
        request.question
    )
    # 2. Return grounded answer to frontend
    return {
        "question": request.question,
        "answer": answer
    }
```

BACKEND ORCHESTRATION



3 steps, one endpoint

The backend sends the question to the Knowledge Base, receives a grounded answer, and returns it to the frontend. The frontend never talks to Bedrock directly.

WHAT THE BACKEND DOES

- ✓ Sends question to Knowledge Base
- ✓ Receives grounded answer
- ✓ Returns response to frontend

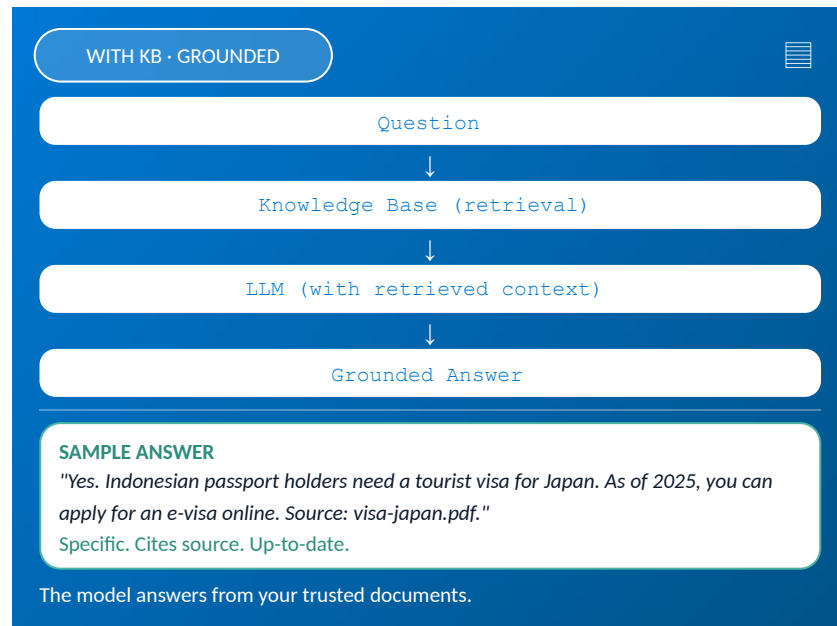
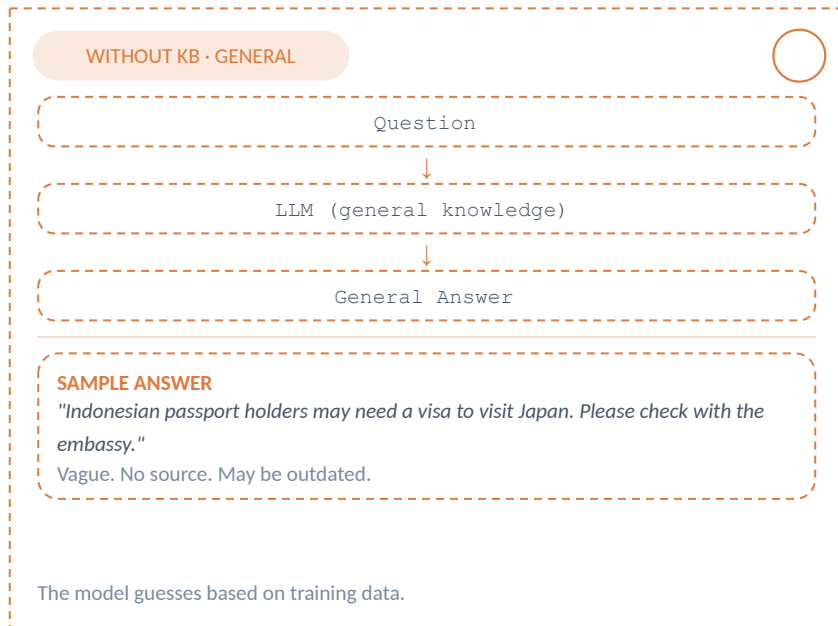
WHY NOT CALL FROM FRONTEND?

Security. AWS credentials live on the backend — never in the browser. The frontend only calls your FastAPI endpoint.

Same pattern as Session 5 — backend owns AI calls.

Part 7 — Compare Results

Same question, two architectures. Which answer would you trust in production?

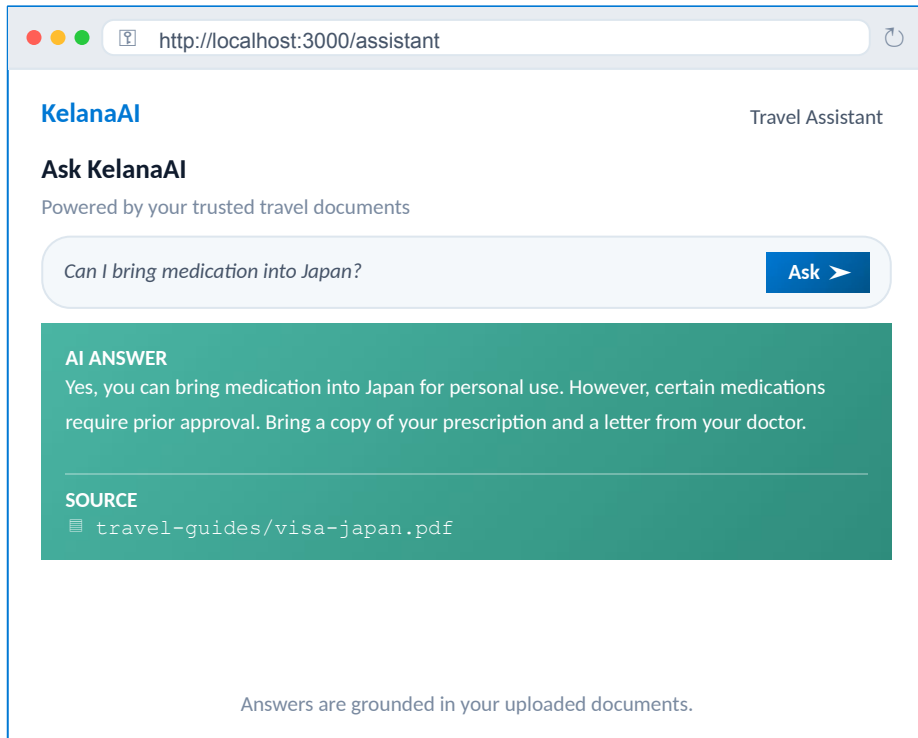


Which answer would you trust in a production application?

RAG answers are specific, citable, and easy to update.

Part 8 — Integrate into Next.js

Build /assistant. Users type travel questions, get grounded answers with source references.



The screenshot shows a web browser at `http://localhost:3000/assistant`. The page header includes the logo "KelanaAI" and the title "Travel Assistant". Below this is a section titled "Ask KelanaAI" with the subtitle "Powered by your trusted travel documents". A text input field contains the question "Can I bring medication into Japan?", followed by a blue "Ask" button with a right-pointing arrow. The answer is displayed in a green box under the heading "AI ANSWER", stating: "Yes, you can bring medication into Japan for personal use. However, certain medications require prior approval. Bring a copy of your prescription and a letter from your doctor." Below the answer, under the heading "SOURCE", is a list icon followed by the text "travel-guides/visa-japan.pdf". At the bottom of the interface, a footer note reads: "Answers are grounded in your uploaded documents."

TRANSFORMATION



From itinerary generator to travel assistant

KelanaAI is no longer just an itinerary generator. Users can ask factual travel questions and get answers grounded in your trusted documents — with source citations.

WHAT USERS SEE

- ✓ AI answer (grounded in docs)
- ✓ Supporting document reference
- ✓ Confidence that the answer is current

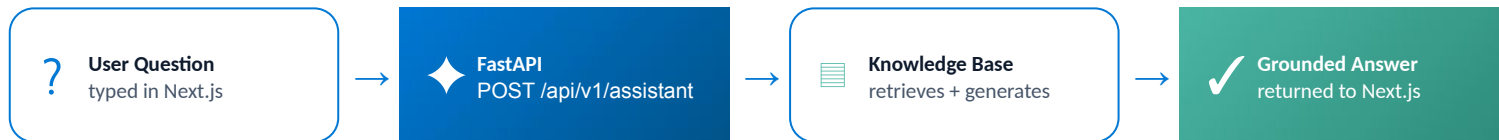
UX NOTE

Showing the source document name builds trust. Users can verify the answer against the original PDF.

This is what enterprise AI assistants look like.

Hands-on Lab — RAG Assistant Endpoint

Build POST /api/v1/assistant. Question → Knowledge Base → Grounded Answer → Next.js.



POST /api/v1/assistant — Request

REQUEST

```
{
  "question": "What documents are required to visit Japan?"
}
```

200 OK — Response

RESPONSE

```
{
  "answer": "According to the uploaded travel guide, you need a valid passport, a tourist visa (or e-visa), and return flight details."
}
```



Build the endpoint. Upload 4 travel PDFs to S3. Test with 5 different questions. Verify answers cite your documents.

Time-box: 30 min

Challenge

Expand the Knowledge Base. Show source citations.

CORE CHALLENGE — EXPAND KB



Upload additional documents

Add more travel guides and test questions that require the new documents.

EXAMPLE DOCUMENTS

- ✓ South Korea Travel Guide
- ✓ Singapore Visa Guide
- ✓ Travel Insurance Policy

THEN

→ Ask questions that require the new documents

Sync the KB after each upload.

BONUS



Show Source Citations

Display the source document name alongside the AI response.

UI HINT

```
<div>
  <p>{answer}</p>
  <small>
    Source: {source}
  </small>
</div>
```

WHY IT MATTERS

Citations build trust. Users can verify answers against the original document.

The Retrieve API returns source references.

Git Checkpoint

Commit today's work. KelanaAI learned how to use trusted knowledge.

```
bash — kelana-ai

# stage today's RAG work
git add .

# commit with a descriptive message
git commit -m "Add Amazon Bedrock Knowledge Base"

# push to GitHub
git push
```



Today KelanaAI learned how to use trusted knowledge.

Tag your commit: `git tag session-9`

Mini Quiz

Quick check — discuss with your neighbour, then we'll review.

Q1

What problem does Retrieval-Augmented Generation solve?

Hint: Think about what the LLM doesn't know.

Q2

Does Amazon Bedrock permanently learn from uploaded documents?

Hint: Retrieve vs train.

Q3

Why is a Knowledge Base more reliable than prompt-only responses?

Hint: Sources and updates.

Q4

When should developers choose RAG over hardcoded business rules?

Hint: Fixed rules vs unstructured knowledge.



Discuss for 5 minutes. We'll review answers together.

Homework & What's Next

Expand the Knowledge Base tonight — and tomorrow, KelanaAI learns to remember conversations.

HOMEWORK — EXPAND THE KB



Add more documents and compare RAG vs base-model answers.

01



Upload 3+ travel documents

PDFs, text files, or Markdown — sync to S3.

02



Test with 5 new questions

Ask questions that require the new documents.

03



Record improved answers

Compare RAG answers vs base-model answers.



Commit and push your solution.

NEXT SESSION — 10

KelanaAI Learns to Remember Conversations

Today KelanaAI learned to answer using trusted documents. However...

"Every conversation still stands alone. If a user asks: What should we do on Day 2? — KelanaAI forgets the previous context."

Tomorrow: Conversational Memory

- Store conversation history in PostgreSQL
- Rebuild prompts using previous messages
- Natural multi-turn AI chat

KelanaAI becomes a true conversational assistant.





MILESTONE ACHIEVED

KelanaAI became an enterprise-ready AI assistant

Instead of relying only on the knowledge embedded within a foundation model, it can now retrieve and use trusted information from your own documents before generating a response.



Foundation Models

Provide reasoning capabilities.



Knowledge Bases

Provide trusted information.



Retrieval

Connects the two at query time.



Your Application

Orchestrates the entire workflow.



This is the foundation of many production AI assistants built on AWS.

Tomorrow: conversational memory. Then deployment. Then graduation.

